🚀 Redis Complete Masterclass

সবগুলো রিয়েল-ওয়ার্ল্ড ইউজ কেস, ইন্টারনাল মেকানিজম এবং আর্কিটেকচারাল ব্যাখ্যা — সহজ বাংলায় সম্পূর্ণ গাইডবুক

13 Core Modules | Production Ready | TypeScript & ioredis | Postman Ready | Video & Interview Guide

📘 Redis Complete Masterclass — সহজ ভাষায় গভীর বিশ্লেষণ (A to Z গাইড)

> এই ডকুমেন্টটি বিশেষভাবে তৈরি করা হয়েছে যাতে Redis-এর প্রতিটি কনসেপ্ট একদম গোড়া থেকে বাস্তব জীবনের উদাহরণের সাহায্যে সহজে বোঝা যায়। আপনি নিজে শেখার জন্য বা ইউটিউব/কোর্সের ভিডিও বানানোর সময় এটি একটি পরিপূর্ণ স্ক্রিপ্ট ও রেফারেন্স হিসেবে ব্যবহার করতে পারবেন।

📑 সূচিপত্র

1. Redis কী এবং সাধারণ ডাটাবেসের সাথে এর পার্থক্য কী?
2. মডিউল ১: OTP & Temporary Tokens (Strings + TTL)
3. মডিউল ২: Response Caching & Invalidation (Cache-Aside Pattern)
4. মডিউল ৩: Atomic Counters & Metrics (INCR / DECR)
5. মডিউল ৪: Redis Hashes (User Profile & Shopping Cart)
6. মডিউল ৫: Redis Lists (Task Queue & Recent Activities)
7. মডিউল ৬: Redis Sets (Online Users & Mutual Interests)
8. মডিউল ৭: Sorted Sets / ZSET (Real-Time Leaderboards & Ranking)
9. মডিউল ৮: API Rate Limiting Middleware (DDoS & Bot Throttling)
10. মডিউল ৯: Distributed Mutex Lock (Double Booking & Concurrency)
11. মডিউল ১০: Redis Pub/Sub (Real-Time Event Broadcasting)
12. মডিউল ১১: Geospatial Queries (Nearby Riders & Distance Calculation)

13. মডিউল ১২: Big Data Analytics (HyperLogLog & Bitmaps)
14. মডিউল ১৩: Pipelines & Atomic Transactions (MULTI / EXEC)

🧠 Redis কী এবং সাধারণ ডাটাবেসের সাথে এর পার্থক্য কী?

১. সহজ বাস্তব জীবনের উদাহরণ:

- **MongoDB / PostgreSQL হলো আপনার বাড়ির চিলেকোঠার আলমারি:** সেখানে লাখ লাখ ফাইল সুন্দর করে সাজানো থাকে (Hard Disk / SSD)। কিন্তু সেখান থেকে একটি ফাইল খুঁজে বের করে এনে পড়তে একটু সময় লাগে (যেমন ২০-১০০ মিলিসেকেন্ড)।
- **Redis হলো আপনার পড়ার টেবিলের ওপর রাখা প্যাড:** যে কাগজগুলো আপনার এই মুহূর্তে সবচেয়ে বেশি দরকার (যেমন কার ওটিপি কত, লাইভ স্কোর কত), সেগুলো আপনি টেবিলের ওপর চোখের সামনে রেখে দেন (RAM - Random Access Memory)। এক পলকেই দেখা যায় (মাত্র ১-২ মিলিসেকেন্ড)!

২. মূল টেকনিক্যাল পার্থক্য:

বৈশিষ্ট্য	সাধারণ ডাটাবেস (SQL / NoSQL)	Redis
স্টোরেজ কোথায় থাকে?	Hard Disk বা SSD তে	কম্পিউটারের র‍্যামে (RAM)
স্পিড (ল্যাটেন্সি)	২০ms থেকে ৩০০ms	০.৫ms থেকে ২ms (বিদ্যুৎ গতি!)
উদ্দেশ্য	স্থায়ী ডাটা স্টোরেজ	ক্যাশিং, লাইভ কাউন্টিং, রিয়েল-টাইম অপারেশন
আর্কিটেকচার	মাল্টি-থ্রেডেড কুয়েরি প্রসেসিং	সিঙ্গেল-থ্রেডেড ইভেন্ট লুপ (কোনো রেস কন্ডিশন নেই)

মডিউল ১: OTP & Temporary Tokens (Strings + TTL)

📂 **কোড ফাইল:** `src/app/modules/otp/otp.service.ts`

১. বাস্তব জীবনের সমস্যা:

ধরা যাক, একজন ইউজার সাইন-আপ করার সময় তার ফোনে একটি ৪ ডিজিটের OTP পাঠালেন। এই OTP-র মেয়াদ মাত্র ৫ মিনিট।

- **সমস্যা:** আপনি যদি এটি MongoDB বা MySQL-এ সেভ করেন, ৫ মিনিট পর এটিকে ডিলিট করবে কে? আপনাকে ব্যাকগ্রাউন্ডে একটি ক্রন জব (Cron Job) চালাতে হবে যা প্রতি মিনিটে ডাটাবেসে স্ক্যান করে পুরনো ওটিপি ডিলিট করবে। এতে ডাটাবেসের ওপর অযথা প্রেশার পড়ে।
- **Redis কীভাবে সমাধান করে?** Redis-এ একটি ফিচার আছে যাকে বলে **TTL (Time-To-Live)**। আপনি Redis-কে বলে দেবেন: *"এই ওটিপি রাখো, ৩০০ সেকেন্ড পর এটা নিজে নিজেই হাওয়া হয়ে যাবে।"* Redis নির্দিষ্ট সময় পর মেমোরি থেকে এটিকে ডিলিট করে দেয়।

২. কোড কীভাবে কাজ করছে?

```
// ১. ওটিপি তৈরি ও সেভ করা (৩০০ সেকেন্ড মেয়াদের জন্য)
await redis.set(`otp:${email}`, otp, "EX", 300);

// ২. ওটিপি খোঁজা ও সময় দেখা
const storedOTP = await redis.get(`otp:${email}`);
const ttl = await redis.ttl(`otp:${email}`);

// ৩. একবার ভেরিফাই হয়ে গেলে ডিলিট করা (One-time use)
await redis.del(`otp:${email}`);
```

- `SET key value EX 300` : `EX` মানে Expiration in seconds। কি তৈরি এবং মেয়াদ সেট—দুটো কাজ একসাথে এক মাইক্রো-সেকেন্ডে ঘটে।
- `TTL key` : এটি দিয়ে জানা যায় আর কত সেকেন্ড বাকি আছে।
- `DEL key` : ইউজার সঠিক ওটিপি দিলে সাথে সাথে ডিলিট করে দেওয়া হয় যাতে একই ওটিপি দিয়ে দ্বিতীয়বার কেউ লগইন না করতে পারে।

🧪 পোস্টম্যানে টেস্ট:

1. `POST /api/otp/send-otp` বডিতে `{ "email": "sajjad@test.com" }` পাঠালে ওটিপি জেনারেট হবে।
2. `POST /api/otp/verify-otp` এ সঠিক ওটিপি দিলে বলবে "OTP verified successfully"। ৫ মিনিট পর দিলে বলবে "OTP expired or not found"।

মডিউল ২: Response Caching & Invalidation (Cache-Aside Pattern)

📂 **কোড ফাইল:** `src/app/modules/cache/cache.service.ts`

১. বাস্তব জীবনের সমস্যা:

আপনার একটি দারাজ বা অ্যামাজনের মতো ই-কমার্স ওয়েবসাইট আছে। হোমপেজে ঢুকলে ১০০০টি প্রোডাক্ট লোড হয়।

- ডাটাবেসে জটিল কুয়েরি (Join, Sort, Aggregation) চালিয়ে ১০০০টি প্রোডাক্ট আনতে সময় লাগে **১৫০০ মিলিসেকেন্ড (১.৫ সেকেন্ড)**।
- যদি প্রতি মিনিটে ১০,০০০ জন ইউজার ওয়েবসাইটে আসে, ডাটাবেসের ওপর বিশাল লোড পড়বে এবং সাইট ক্র্যাশ করবে।

২. Cache-Aside প্যাটার্ন কী?

1. **প্রথম রিকোয়েস্ট (Cache Miss):** ইউজার আসল। Redis চেক করে দেখা গেল কিচ্ছু নেই। তখন ডাটাবেস থেকে আনতে ১.৫ সেকেন্ড লাগল। আনার পর আমরা রেজাল্টটি Redis-এ ৬০ সেকেন্ডের জন্য ক্যাশ করে রাখলাম।
2. **দ্বিতীয় রিকোয়েস্ট (Cache Hit):** দ্বিতীয় ইউজার আসল। এবার আর ডাটাবেসে যাওয়ার দরকার নেই! Redis থেকে ডাটা নিয়ে আসতে সময় লাগবে মাত্র **১-২ মিলিসেকেন্ড**!
3. **Cache Invalidation (ক্যাশ মোছা):** অ্যাডমিন যদি নতুন একটি প্রোডাক্ট যোগ করে, তখন পুরনো ক্যাশ রেখে দিলে ইউজার নতুন প্রোডাক্ট দেখতে পাবে না। তাই নতুন প্রোডাক্ট সেভ হওয়ার সাথে সাথে `redis.del("cache:products:all")` কল করে ক্যাশ মুছে ফেলা হয়।

৩. কোড বিশ্লেষণ:

```
// ১. আগে ক্যাশ চেক করি
const cachedData = await redis.get(CACHE_KEY);
if (cachedData) {
  return JSON.parse(cachedData); // ⚡ ১ মিলিসেকেন্ডে রেসপন্স!
}

// ২. ক্যাশে না থাকলে আসল DB থেকে আনি
const products = await fetchFromDB();

// ৩. পরের বারের জন্য Redis-এ ৬০ সেকেন্ডের জন্য রেখে দিই
await redis.set(CACHE_KEY, JSON.stringify(products), "EX", 60);
```

💉 পোস্টম্যানে টেস্ট:

1. `GET /api/cache/products` প্রথমবার রান করলে রেসপন্সে দেখতে পাবেন: `source: "DATABASE", responseTime: "1504 ms"`।
2. সাথে সাথে আবার সেন্ড করুন! দেখতে পাবেন: `source: "REDIS_CACHE", responseTime: "2 ms"`! চোখের পলকে রেজাল্ট চলে আসবে।

মডিউল ৩: Atomic Counters & Metrics (INCR / DECR)

📂 **কোড রেফারেন্স:** `src/app/modules/counter/counter.service.ts`

১. বাস্তব জীবনের সমস্যা (Race Condition):

ধরা যাক ইউটিউবে একটি ভিডিওর ভিউ কাউন্ট ১০০ আছে।

- একই সাথে দুজন মানুষ ভিডিওটি প্লে করল।
- ডাটাবেস কোড যদি সাধারণ নিয়মে চলে:
  - ইউজার ১ ডাটাবেস থেকে পড়ল: ভিউ ১০০।
  - ইউজার ২ ডাটাবেস থেকে পড়ল: ভিউ ১০০।
  - ইউজার ১ ১০০ এর সাথে ১ যোগ করে ডাটাবেসে ১০১ সেভ করল।
  - ইউজার ২-ও ১০০ এর সাথে ১ যোগ করে ডাটাবেসে ১০১ সেভ করল।
- **ফলাফল:** দুজন মানুষ ভিডিও দেখলেও ভিউ বাড়ল মাত্র ১টি! একে বলে **Race Condition বা Concurrency Bug**।

২. Redis কীভাবে সমাধান করে?

Redis ইঞ্জিন হলো **সিঙ্গেল-থ্রেডেড (Single-Threaded)**। তার মানে Redis-এর কাছে যদি একসাথে ১০০০টি ইনক্রিমেন্ট রিকোয়েস্ট আসে, Redis সেগুলোকে সিরিয়ালি একটার পর একটা সাজিয়ে নেয়।

- `INCR` কমান্ড কোনো রিড-রাইট ঝামেলা ছাড়াই মেমোরিতে সরাসরি ১ বাড়িয়ে দেয়।
- ১০,০০০ মানুষ একই মিলি-সেকেন্ডে ক্লিক করলেও ১টি ভিউ বা লাইকও মিস হবে না!

```
// পেজ ভিউ বা লাইক ১ বাড়ানো
const views = await redis.incr(`counter:pageview:${pageId}`);

// আনলাইক করলে ১ কমানো
const likes = await redis.decr(`counter:post:${postId}:likes`);
```

মডিউল 8: Redis Hashes (User Profile & Shopping Cart)

📂 **কোড ফাইল:** `src/app/modules/hash/hash.service.ts`

১. String বনাম Hash (কেন আমরা Hash ব্যবহার করব?)

ধরা যাক ইউজারের প্রোফাইল এমন:
`{ name: "Sajjad", age: 25, role: "developer", city: "Dhaka" }`

- **JSON String হিসেবে রাখলে সমস্যা:**
  আপনি যদি শুধু ইউজারের বয়স ২৫ থেকে ২৬ করতে চান, আপনাকে পুরো অবজেক্টটি ডাউনলোড করতে হবে -> `JSON.parse` করতে হবে -> age পরিবর্তন করতে হবে -> আবার `JSON.stringify` করে পুরোটা আপলোড করতে হবে। ছোট একটি কাজের জন্য বিশাল প্রসেসিং!
- **Redis Hash-এর সুবিধা:**
  Redis Hash হলো একটি অবজেক্টের মতো ডিকশনারি। আপনি চাইলে পুরো অবজেক্ট না ছুঁয়ে শুধুমাত্র একটি ফিল্ড আপডেট করতে পারেন:

  ```
  HSET user:101 age 26
  ```

  মাত্র ১টি ফিল্ড আপডেট হয়ে যাবে, বাকি ফিল্ডগুলো অপরিবর্তিত থাকবে।

২. শপিং কার্টের উদাহরণ:

```
// কার্টে ১টি ম্যাকবুক যোগ করা (HINCRBY)
await redis.hincrby(`cart:${userId}`, "macbook_m3", 1);

// ইউজারের পুরো কার্ট দেখা
const cart = await redis.hgetall(`cart:${userId}`);
```

ইউজার যতবার "Add to Cart" চাপবে, `HINCRBY` কোয়ান্টিটি ১ করে বাড়িয়ে দেবে। কোয়ান্টিটি ০ হয়ে গেলে `HDEL` দিয়ে প্রোডাক্টটি কার্ট থেকে ফেলে দেওয়া হয়।

মডিউল ৫: Redis Lists (Task Queue & Recent Activities)

📂 **কোড ফাইল:** `src/app/modules/list/list.service.ts`

১. বাস্তব জীবনের সমস্যা:

ইউজার সাইন আপ করার পর তাকে একটি ওয়েলকাম ইমেইল পাঠাতে হবে। ইমেইল পাঠাতে ৩ সেকেন্ড সময় লাগে।

- ইউজার যদি সাইন-আপ বাটনে ক্লিক করে ৩ সেকেন্ড লোডিং স্ক্রিন দেখে বসে থাকে, তবে ইউজারের অভিজ্ঞতা খুব খারাপ হবে।
- **সমাধান:** আমরা সাইন-আপ শেষ করেই একটি টাস্ক Redis লিস্টে ফেলে দেব (`LPUSH`), আর ইউজারকে সাথে সাথে "Success" রেসপন্স দিয়ে দেব। ব্যাকগ্রাউন্ডে একটি ওয়ার্কার প্রসেস শান্তিমতো লিস্ট থেকে টাস্ক তুলে তুলে (`RPOP`) ইমেইল পাঠাবে। একে বলে **Producer-Consumer Architecture** বা **Background Queue**।

২. কোড বিশ্লেষণ:

```
// Producer: কিউ-তে টাস্ক পুশ করা (বাঁ দিক দিয়ে ঢোকানো)
await redis.lpush("queue:emails", JSON.stringify({ to: "user@test.com", subjec

// Consumer: কিউ থেকে কাজ বের করা (ডান দিক দিয়ে বের করা - FIFO)
const task = await redis.rpop("queue:emails");
```

৩. রিসেন্ট অ্যাক্টিভিটি ক্যাপিং (`LTRIM` এর জাদু):

ফেসবুকে আপনার শেষ ১০টি নোটিফিকেশন দেখাতে চান। ডাটাবেসে কি কোটি কোটি নোটিফিকেশন জমা করে রাখবেন?

```
await redis.lpush(`activities:user:${userId}`, newActivity);
await redis.ltrim(`activities:user:${userId}`, 0, 9); // শুধুমাত্র ০ থেকে ৯ (১০টি)
```

`LTRIM` নিশ্চিত করে লিস্টের সাইজ কখনো ১০টির বেশি হবে না। পুরনো ডাটা নিজে থেকেই মেমোরি থেকে মুছে যায়।

মডিউল ৬: Redis Sets (Online Users & Mutual Interests)

📂 **কোড ফাইল:** `src/app/modules/set/set.service.ts`

১. Set-এর বিশেষত্ব কী?

- সেটে কখনো **ডুপ্লিকেট ভ্যালু** থাকতে পারে না (সব উপাদান ইউনিক)।
- কোনো সদস্য সেটে আছে কি না তা চেক করা (`SISMEMBER`) মাত্র $O(1)$ সময় নেয়। অর্থাৎ ১ কোটি ইউজারের মধ্যেও একজন ইউজার অনলাইনে আছে কি না তা ১ মিলিসেকেন্ডেই বলে দেওয়া যায়।

২. অনলাইন ইউজার ট্র্যাকিং:

```
// ইউজার কানেক্ট হলে সেটে যোগ করি
await redis.sadd("app:online_users", userId);

// ইউজার ডিসকানেক্ট হলে সেট থেকে বাদ দিই
await redis.srem("app:online_users", userId);

// ইউজার অনলাইন আছে কি না চেক করি (O(1))
const isOnline = await redis.sismember("app:online_users", userId);
```

৩. মিউচুয়াল ফ্রেন্ড / কমন ইন্টারেস্ট (`SINTER`):

- ইউজার ১-এর পছন্দ: `["coding", "redis", "football", "chess"]`
- ইউজার ২-এর পছন্দ: `["gaming", "redis", "football", "movies"]`
- দুজনের মধ্যে মিল কোথায়?

```
// Redis Command: SINTER
const common = await redis.sinter("user:1:interests", "user:2:interests");
// আউটপুট: ["redis", "football"]
```

ডাটাবেসে বিশাল জয়েন কুয়েরি না চালিয়ে এক লাইনে কমন বিষয়গুলো বের হয়ে আসে।

মডিউল ৭: Sorted Sets / ZSET (Real-Time Leaderboards & Ranking)

📂 **কোড ফাইল:** `src/app/modules/leaderboard/leaderboard.service.ts`

১. কেন সাধারণ ডাটাবেসে লিডারবোর্ড বানানো কঠিন?

পাবজি বা ফ্রি-ফায়ারের মতো গেমে প্রতি সেকেন্ডে হাজার হাজার প্লেয়ার পয়েন্ট পাচ্ছে।

- SQL-এ টপ ১০ প্লেয়ার বের করতে হলে লিখতে হয়: `SELECT * FROM players ORDER BY score DESC LIMIT 10`।
- ১০ লাখ প্লেয়ারের টেবিল যদি প্রতি সেকেন্ডে হাজার বার সর্ট করতে হয়, সার্ভার ক্র্যাশ করবে।

২. Redis ZSET কীভাবে কাজ করে?

ZSET-এ প্রতিটি নামের সাথে একটি **Score** যুক্ত থাকে। ডেটা ঢোকানোর সাথে সাথেই Redis একে স্বয়ংক্রিয়ভাবে স্কোরের ক্রমানুসারে সাজিয়ে ফেলে (ইন্টারনালি Skip List ব্যবহার করে)।

```
// প্লেয়ারের স্কোর সেট করা
await redis.zadd("leaderboard:weekly", 1500, "Sajjad");

// প্লেয়ার নতুন ৫০ পয়েন্ট পেল (ZINCRBY)
await redis.zincrby("leaderboard:weekly", 50, "Sajjad");

// টপ ১০ জন হাইয়েস্ট স্কোরার বের করা
const top10 = await redis.zrevrange("leaderboard:weekly", 0, 9, "WITHSCORES");

// সাজ্জাদের বর্তমান র‍্যাংক কত? (১ম, ২য় নাকি ৫০০তম?)
const rank = await redis.zrevrank("leaderboard:weekly", "Sajjad");
```

১০ লাখ ইউজারের মধ্যেও র‍্যাংক বের করতে কয়েক মাইক্রো-সেকেন্ড সময় লাগে!

মডিউল ৮: API Rate Limiting Middleware (DDoS & Bot Throttling)

📂 **কোড ফাইল:** `src/app/modules/rateLimiter/rateLimiter.service.ts`

১. বাস্তব জীবনের সমস্যা:

কোনো হ্যাকার স্ক্রিপ্ট লিখে আপনার লগইন এপিআই-তে প্রতি সেকেন্ডে ১০০০ বার ভুল পাসওয়ার্ড পাঠিয়ে সার্ভার ডাউন করে দিতে পারে।

- **সমাধান:** রেট লিমিটার নিয়ম করে দেবে: *"একই আইপি থেকে ৩০ সেকেন্ডে সর্বোচ্চ ৫টির বেশি রিকোয়েস্ট পাঠানো যাবে না।"* ৬ নম্বর রিকোয়েস্টে সার্ভার বলবে `429 Too Many Requests`।

২. কীভাবে কাজ করে?

```
const key = `rl:${clientIp}`;
const currentRequests = await redis.incr(key);

// প্রথম রিকোয়েস্টে ৩০ সেকেন্ডের মেয়াদ শুরু
if (currentRequests === 1) {
  await redis.expire(key, 30);
}

// লিমিট অতিক্রম করলে রিকোয়েস্ট আটকে দেওয়া
if (currentRequests > 5) {
  return res.status(429).json({ message: "Too many requests! Try again later."
}
```

মডিউল ৯: Distributed Mutex Lock (Double Booking & Concurrency)

📂 **কোড ফাইল:** `src/app/modules/lock/lock.service.ts`

১. সিনেমার শেষ সিট ও Double Booking সমস্যা:

সিনেমার মাত্র ১টি সিট খালি আছে (Seat A1)।

- রাত ১২:০০:০০ টায় রহিম এবং করিম একই সাথে "বুক করুন" বাটনে ক্লিক করল।
- সাধারণ কোডে রহিম ও করিম দুজনের কাছেই সিট খালি দেখাবে এবং দুজনের ব্যাংক থেকেই টাকা কেটে সিট বুক হয়ে যাবে! একে বলে **Double Booking**।

২. Redis ডিস্ট্রিবিউটেড লক কীভাবে সমাধান করে?

টিকিট বুকিংয়ের কাজ শুরু করার আগে একটি **তালা (Lock)** লাগাতে হয়:

```
// SET key token NX PX 5000
// NX মানে "Only if Not Exists" (যদি তালা না থাকে তবেই তালা লাগাও)
// PX 5000 মানে ৫ সেকেন্ড পর তালা নিজে থেকেই খুলে যাবে (যাতে কোনো ডেডলক না হয়)
const acquired = await redis.set("lock:ticket:A1", uniqueToken, "PX", 5000, "N

if (!acquired) {
  return res.status(409).json({ message: "অন্য কেউ এই টিকিটটি বুক করছে! কিছুক্ষণ
}

try {
  // ১.৫ সেকেন্ড সময় নিয়ে পেমেন্ট ও টিকিট বুক সম্পন্ন করি
  await processPayment();
} finally {
  // কাজ শেষ! তালা নিরাপদে খুলে দেওয়া হলো
  await releaseLockWithLuaScript();
}
```

রহিম ও করিম একসাথে আসলেও Redis শুধুমাত্র একজনকে লক দেবে, অন্যজনকে সাথে সাথে ফিরিয়ে দেবে।

মডিউল ১০: Redis Pub/Sub (Real-Time Event Broadcasting)

📂 **কোড ফাইল:** `src/app/modules/pubsub/pubsub.service.ts`

১. বাস্তব জীবনের উদাহরণ:

একটি রেডিও স্টেশনের কথা ভাবুন। রেডিও জকি মাইকে কথা বলে (Publisher)। সে জানে না ঢাকার শহরে কে কে রেডিও শুনছে। যে যে রেডিও অন করে সেই চ্যানেলে টিউন করে রেখেছে (Subscriber), তারা সবাই মুহূর্তে আওয়াজ শুনতে পায়।

২. ব্যাকএন্ডে কেন লাগে?

আপনার ব্যাকএন্ড সার্ভার ৪টি মেশিনে চলছে। ইউজার ১ মেশিন-এ তে অর্ডার প্লেস করল। কিন্তু ডেলিভারি বয়ের অ্যাপ কানেক্টেড মেশিন-ডি এর সাথে।

- মেশিন-এ Redis চ্যানেলে পাবলিশ করে দেবে: `PUBLISH orders "New Order #101"`
- সব সার্ভার সাথে সাথে নোটিফিকেশন পেয়ে যাবে এবং যার যার সকেটে মেসেজ পাঠিয়ে দেবে।

```
// মেসেজ ব্রডকাস্ট করা
await redis.publish("notifications:live", JSON.stringify({ event: "NEW_ORDER",

// ব্যাকগ্রাউন্ডে শোনা (Subscriber)
redisSub.subscribe("notifications:live");
redisSub.on("message", (channel, message) => {
  console.log("নতুন মেসেজ পেয়েছি:", message);
});
```

মডিউল ১১: Geospatial Queries (Nearby Riders & Distance Calculation)

📂 **কোড ফাইল:** `src/app/modules/geo/geo.service.ts`

১. বাস্তব জীবনের সমস্যা:

পাঠাও বা উবার অ্যাপে আপনি যখন রাইড চান, অ্যাপ কীভাবে ১ সেকেন্ডের মধ্যে বলে দেয় আপনার আশেপাশে ৫ কিলোমিটারের মধ্যে কোন কোন বাইকার আছে?

- ডাটাবেসে লাখ লাখ বাইকারের ল্যাটিটিউড ও লংগিটিউড দিয়ে গোলীয় ত্রিকোণমিতিক হিসাব (Haversine formula) করতে গেলে প্রচুর সময় লাগে।

২. Redis কীভাবে সমাধান করে?

Redis আপনার স্থানাঙ্কগুলোকে **Geohash** অ্যালগরিদম দিয়ে ৫২-বিট নাম্বারে রূপান্তর করে মেমোরিতে সাজিয়ে রাখে।

```
// ১. রাইডারের লোকেশন সেভ করা (মনে রাখবেন: আগে Longitude, পরে Latitude)
await redis.geoadd("locations:riders", 90.4152, 23.7925, "rider_gulshan");
await redis.geoadd("locations:riders", 90.4043, 23.7937, "rider_banani");

// ২. দুই রাইডারের মধ্যবর্তী দূরত্ব কত কিলোমিটার?
const dist = await redis.geodist("locations:riders", "rider_gulshan", "rider_b

// ৩. আমার লোকেশনের আশেপাশে ৫ কিলোমিটারের মধ্যে কারা আছে?
const nearby = await redis.geosearch("locations:riders", "FROMLONLAT", 90.4125
```

কয়েক মিলিসেকেন্ডেই আশেপাশের সব রাইডারের লিস্ট তাদের দূরত্বের মাপসহ পাওয়া যায়।

মডিউল ১২: Big Data Analytics (HyperLogLog & Bitmaps)

📂 **কোড ফাইল:** `src/app/modules/analytics/analytics.service.ts`

১. HyperLogLog (HLL) — মেমোরির ম্যাজিক:

ধরা যাক আপনার প্রথম আলো বা বিডিনিউজ২৪-এর মতো সাইটে প্রতিদিন ১ কোটি মানুষ আসে। আপনি দেখতে চান আজকে মোট **কতজন ইউনিক ভিজিটর** এসেছে।

- **সাধারণ সমস্যা:** ১ কোটি আইপি এড্রেস যদি সেটে (`Set`) রাখেন, আপনার প্রায় **৮০০ মেগাবাইট থেকে ১ জিবি র‍্যাম** শেষ হয়ে যাবে!
- **HyperLogLog এর ম্যাজিক:** এটি একটি গাণিতিক অ্যালগরিদম। এতে আপনি ১০ জন ভিজিটর দেন বা ১০ কোটি ভিজিটর দেন—এটি **সর্বোচ্চ মাত্র ১২ কিলোবাইট (KB)** মেমোরি খরচ করে! এবং মাত্র ০.৮১% ভুলের ব্যবধানে নিখুঁতভাবে ইউনিক সংখ্যা বলে দেয়!

```
// ইউনিক ভিজিটর কাউন্ট করা
await redis.pfadd("hll:visitors:home", userIp);
const totalUnique = await redis.pfcount("hll:visitors:home");
```

২. Bitmaps (দৈনিক উপস্থিতি / DAU ট্র্যাকার):

- প্রতিটি ইউজারের জন্য একটি করে ছোট বিট (০ অথবা ১)।
- ইউজার ৪২ নম্বর যদি আজকে অ্যাপে ঢুকে, ৪২ নম্বর বিটটি ১ হয়ে যাবে:

```
// ইউজারের উপস্থিতি ১ করা
await redis.setbit("dau:2026-10-03", 42, 1);

// আজকে মোট কতজন ইউজার এক্টিভ ছিল?
const totalActive = await redis.bitcount("dau:2026-10-03");

// ইউজার ৪২ কি আজকে ঢুকেছিল?
const wasActive = await redis.getbit("dau:2026-10-03", 42); // রিটার্ন করে 1 অথব
```

১০ লাখ ইউজারের আজকের উপস্থিতি স্টোর করতে মাত্র ১২৫ KB র‍্যাম লাগে!

মডিউল ১৩: Pipelines & Atomic Transactions (MULTI / EXEC)

📂 **কোড ফাইল:** `src/app/modules/batch/batch.service.ts`

১. Redis Pipelining (নেটওয়ার্ক স্পিড বুস্টার):

ধরা যাক আপনাকে ১০০টি প্রোডাক্ট Redis-এ সেভ করতে হবে।

- **সাধারণ নিয়ম:** ১টি পাঠাবেন -> রেসপন্স আসবে -> ২য়টি পাঠাবেন -> রেসপন্স আসবে। এতে ১০০ বার ইন্টারনেটে যাওয়া-আসা (Round Trip) করতে গিয়ে ২০০-৩০০ মিলিসেকেন্ড লেগে যায়।
- **Pipeline:** আপনি ১০০টি কমান্ড একসাথে একটিমাত্র প্যাকেটে ভরে Redis-এ পাঠিয়ে দেবেন। Redis একবারে ১০০টি রান করে একবারে রেজাল্ট ফেরত দেবে। সময় ২০০ms থেকে কমে মাত্র **২-৩ মিলিসেকেন্ডে** নেমে আসে!

```
const pipeline = redis.pipeline();
for (let i = 1; i <= 100; i++) {
  pipeline.set(`item:${i}`, `value_${i}`);
}
await pipeline.exec(); // এক ঝটকায় সব এক্সিকিউট!
```

২. Transactions (`MULTI` / `EXEC`):

ধরা যাক আপনি বিকাশ বা ব্যাংকে টাকা ট্রান্সফার করবেন।

- ইউজার 'A' এর একাউন্ট থেকে ১০০ টাকা মাইনাস হবে।
- ইউজার 'B' এর একাউন্টে ১০০ টাকা প্লাস হবে।
- এমন যেন কখনো না হয় যে A-এর টাকা কেটে গেল, কিন্তু সার্ভার ক্র্যাশ করায় B টাকা পেল না!
- `MULTI ... EXEC` নিশ্চিত করে এই দুটি কাজ একটি সিঙ্গেল ব্লকে একসাথেই ঘটবে, মাঝখানে অন্য কেউ ঢুকতে পারবে না।

```
const transaction = redis.multi();
transaction.decrby("wallet:user:A", 100);
transaction.incrby("wallet:user:B", 100);
await transaction.exec();
```

🎯 ভিডিও বানানোর জন্য আপনার চেকলিস্ট (Summary Flow):

1. **শুরু করুন:** "Redis মানে শুধুই ক্যাশিং নয়, এটি একটি পাওয়ারফুল ইন-মেমোরি ডেটা স্ট্রাকচার স্টোর।"
2. **ওটিপি ও স্ট্রিংস দেখান:** TTL কিভাবে ডেভলপারের জীবন সহজ করে।
3. **ক্যাশিং দেখান:** পোস্টম্যানে ১৫০০ms বনাম ২ms এর লাইভ স্পিড পার্থক্য দেখান।

4. **হ্যাশ বনাম স্ট্রিং:** কেন সম্পূর্ণ অবজেক্ট বারবার আপডেট করার চেয়ে হ্যাশ অনেক বেশি এফিশিয়েন্ট।
5. **কিউ ও লিডারবোর্ড:** ব্যাকগ্রাউন্ড কাজ এবং গেমিং স্কোরবোর্ড কীভাবে এক নিমিষে কাজ করে।
6. **সিকিউরিটি ও সেফটি:** রেট লিমিটিং এবং ডাবল-বুকিং বন্ধে ডিসট্রিবিউটেড লকিং।
7. **ম্যাজিক ফিচার্স:** HyperLogLog (১২ KB ফিক্সড মেমোরি) ও Bitmaps বুঝিয়ে সবাইকে চমকে দিন!

এই পূর্ণাঙ্গ ডকুমেন্টটি আপনার প্রজেক্টের রুট ডিরেক্টরিতে `REDIS_MASTERCLASS_BANGLA_DOC.md` হিসেবে স্থায়ীভাবে সেভ করা হয়েছে। আপনি যখনই ভিডিও রেকর্ড করবেন বা রিভিশন দেবেন, এটি ওপেন করে পয়েন্ট বাই পয়েন্ট সহজে ব্যাখ্যা করতে পারবেন! 🚀